

Clase Celda — Representa cada casilla del tablero

```
public class Celda {  
  
    private String tipo;  
    private boolean visitada;  
  
    public Celda(String tipo) {  
        this.tipo = tipo;  
        this.visitada = false;  
    }  
    public String getTipo() {  
        return tipo;  
    }  
  
    public void setTipo(String tipo) {  
        this.tipo = tipo;  
    }  
  
    public boolean isVisitada() {  
        return visitada;  
    }  
  
    public void setVisitada(boolean visitada) {  
        this.visitada = visitada;  
    }  
  
    // Te amo profe  
    public String toString() {  
        if (tipo.equals("inicio")) return "[ I ]";  
        if (tipo.equals("objetivo")) return "[ X ]";  
        if (tipo.equals("potenciador")) return "[ P ]";  
        if (tipo.equals("modificador")) return "[ M ]";  
        return "[   ]";  
    }  
}
```

Atributos privados: 'tipo' guarda si la celda es inicio, objetivo, etc. 'visitada' indica si el jugador ya

pasó por la celda. Recibe el tipo de celda y pone visitada en false por defecto. getTipo() / setTipo(): getters y setters para leer o cambiar el tipo de la celda.

isVisitada() / setVisitada(): permiten saber si el jugador ya pisó esta celda, y marca como visitada.

toString(): convierte la celda a un símbolo visual: [I]=inicio, [X]=objetivo, [P]=potenciador, [M]=modificador, []=vacía.

Clase Jugador — Controla al personaje y su movimiento

```
public class Jugador {  
    private int fila;  
    private int col;  
    private int puntaje;  
  
    public Jugador() {  
        this.fila = 0;  
        this.col = 0;  
        this.puntaje = 0;  
    }  
    public boolean mover(String direccion) {  
        int nuevaFila = fila;  
        int nuevaCol = col;  
  
        if (direccion.equals("arriba"))        nuevaFila--;  
        else if (direccion.equals("abajo"))    nuevaFila++;  
        else if (direccion.equals("izquierda")) nuevaCol--;  
        else if (direccion.equals("derecha"))  nuevaCol++;  
        else {  
            System.out.println("Direccion no reconocida");  
            return false;  
        }  
        if (nuevaFila < 0 || nuevaFila >= 4 || nuevaCol < 0 || nuevaCol >= 4) {  
            System.out.println("No puedes moverte en esa direccion, te salis del tablero!");  
            return false;  
        }  
        fila = nuevaFila;  
        col = nuevaCol;  
        return true;  
    }  
    public int getFila() { return fila; }  
    public int getCol() { return col; }  
  
    public int getPuntaje() { return puntaje; }  
    public void setPuntaje(int puntaje) { this.puntaje = puntaje; }  
}
```

Atributos privados: fila y col guardan la posición del jugador en el tablero. puntaje acumula los puntos obtenidos.

Constructor: inicializa al jugador en la posición (0,0) con 0 puntos.

mover(direccion): calcula la nueva posición según la dirección recibida (arriba/abajo/izquierda/derecha).

Validación de límites: si la nueva posición está fuera del tablero (fila/col < 0 o >= 4) el movimiento se cancela y devuelve false.

Si el movimiento es válido, actualiza fila y col, y devuelve true para avisar que se movió.

Getters y setters: getFila(), getCol(), getPuntaje() y setPuntaje() para acceder o modificar los datos del jugador.

Clase tabla — El tablero de juego 4x4

```
public class tabla {  
    private Celda[][] grilla;  
    private int tamaño = 4;  
  
    public tabla() {  
        this.grilla = new Celda[tamaño][tamaño];  
        generarTablero();  
    }  
  
    private void generarTablero() {  
        for (int i = 0; i < tamaño; i++) {  
            for (int j = 0; j < tamaño; j++) {  
                grilla[i][j] = new Celda("vacía");  
            }  
        }  
  
        // Celdas profeas aca estan las celdas profeeeeeeeeeeee miralaaaaasss xd  
        grilla[0][0].setTipo("inicio");  
        grilla[1][2].setTipo("objetivo");  
        grilla[2][0].setTipo("potenciador");  
        grilla[3][1].setTipo("potenciador");  
        grilla[2][3].setTipo("modificador");  
        grilla[3][3].setTipo("modificador");  
    }  
  
    public void mostrar(int jugadorFila, int jugadorCol) {  
        System.out.println();  
        System.out.print("    ");  
        for (int c = 0; c < tamaño; c++) {  
            System.out.print("  " + c + "  ");  
        }  
        System.out.println();  
  
        for (int i = 0; i < tamaño; i++) {  
            System.out.print("  " + i + "  ");  
            for (int j = 0; j < tamaño; j++) {  
                if (i == jugadorFila && j == jugadorCol) {  
                    System.out.print("[ T ] ");  
                } else {  
                    System.out.print(grilla[i][j].toString() + " ");  
                }  
            }  
            System.out.println();  
        }  
  
        System.out.println();  
        System.out.println("Info: [T] jugador [I] inicio [x] objetivo [P] potenciador [M] modificador");  
        System.out.println();  
    }  
  
    public Celda getCelda(int fila, int col) {  
        return grilla[fila][col];  
    }  
}
```

Atributos: grilla es la matriz 2D de Celdas. tamaño define que el tablero es de 4x4.

Constructor tabla(): crea la matriz de Celdas y llama a generarTablero() para inicializarla.

generarTablero(): recorre toda la matriz con dos for anidados y crea una Celda vacía en cada posición.

Configuración del mapa: después de llenar de vacías, se asignan tipos especiales a celdas concretas (inicio, objetivo, potenciadores, modificadores).

mostrar(): imprime el tablero en consola. Primero muestra los números de columna como cabecera.

Dentro del for de filas: si la celda es donde está el jugador muestra [T], sino muestra el símbolo de la celda (toString()).

Leyenda: al final del tablero imprime que significa cada símbolo para que el jugador sepa como leerlo.

getCelda(fila, col): devuelve la celda en esa posición, usada por Main para saber en qué celda cayó el jugador.

Clase Main — Logica principal del juego

```
import java.util.Scanner;
public class Main {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Fabian: Thiago quien es mas lindo Ortiz o Facundo? ");
        System.out.println("Thiago: ");
        System.out.println("a-      a-a-a-      a-a-a-a-a-      a-a-a-a-a-      a-a-a-a-      a-a-a-a-
            a-      a-      a-      a-      a-      a-      a-      a-      a-      \n" +
            "a-      a-      a-      a-a-a-a-      a-      a-      a-      a-      a-a-a-a-      a-a-a-a-      \n" +
            "a-      a-      a-      a-      a-      a-      a-      a-      a-      a-      \n" +
            "a-a-a-a-a-a-a-      a-a-a-a-      a-a-a-a-a-      a-a-a-a-a-      a-a-a-a-a-");
        tabla tablero = new Tabla();
        Jugador jugador = new Jugador();
        boolean jugando = true;

        System.out.println(" ");
        tablero.mostrar(jugador.getFila(), jugador.getCol());

        while (jugando) {
            System.out.println("Posicion actual: [" + jugador.getFila() + "," + jugador.getCol() + "]");
            jugador.getPosicion();

            boolean seMov = false;

            System.out.print("W=arriba S=abajo A=izquierda D=derecha: ");
            String tecla = scanner.nextLine().trim().toLowerCase();

            String dir = "";
            if (tecla.equals("w")) dir = "arriba";
            else if (tecla.equals("s")) dir = "abajo";
            else if (tecla.equals("a")) dir = "izquierda";
            else if (tecla.equals("d")) dir = "derecha";
            else System.out.println("Tecla invalida.");

            if (!dir.isEmpty()) {
                seMov = jugador.mover(dir);
            }

            if (seMov) {
                Celda celdaActual = tablero.getCelda(jugador.getFila(), jugador.getCol());

                if (celdaActual.getTipo().equals("objetivo")) {
                    jugador.setPuntaje(jugador.getPuntaje() + 100);
                    tablero.mostrar(jugador.getFila(), jugador.getCol());
                    System.out.println("LO LOGRASTE SOS LA CABRA");
                    System.out.println("Puntaje final: " + jugador.getPuntaje());
                    jugando = false;
                }
                else if (celdaActual.getTipo().equals("potenciador") && !celdaActual.isVisitada()) {
                    jugador.setPuntaje(jugador.getPuntaje() * 2);
                    celdaActual.setVisitada(true);
                    System.out.println("Potenciador de aura Puntaje duplicado -> " + jugador.getPuntaje());
                    tablero.mostrar(jugador.getFila(), jugador.getCol());
                }
                else if (celdaActual.getTipo().equals("modificador") && !celdaActual.isVisitada()) {
                    jugador.setPuntaje(jugador.getPuntaje() + 50);
                    celdaActual.setVisitada(true);
                    System.out.println("Modificador de belleza +50 puntos " + jugador.getPuntaje());
                    tablero.mostrar(jugador.getFila(), jugador.getCol());
                }
                else {
                    tablero.mostrar(jugador.getFila(), jugador.getCol());
                }
            }
        }

        scanner.close();
    }
}
```

Importacion de Scanner para leer la

Senet rcardea nd elol st eocblajedto sd perl iunscupaarlieo
tablero (el mapa), jugador (el
personaje) y jugando (controla si el
juego sigue).

Se muestra el tablero inicial con la posición del jugador antes de empezar a jugar.

Bucle principal while(jugando): el juego sigue corriendo hasta que jugando pase a false (al ganar o perder).

Se lee la tecla ingresada y se convierte a minúscula. Según la tecla (w/s/a/d) se asigna la dirección correspondiente.

Si seMov es true (el jugador se movio), se obtiene la celda actual y se evalua su tipo.

Si la celda es 'objetivo': suma 100 puntos, muestra el tablero, imprime mensaje de victoria y termina el juego (jugando=false).

Si es 'potenciador' y no fue visitada:
 duplica el puntaje actual y marca la
 celda como visitada.

Si es 'modificador' y no fue visitada:
suma 50 puntos y marca la celda como visitada.

Si es cualquier otra celda: solo muestra el tablero actualizado sin cambiar el puntaje.